Nextjs - CRUD

```
app/
 └── student/
      ├── page.tsx          → GET ALL
      ├── create/
      │     └── page.tsx    → CREATE
      ├── [id]/
      │     └── page.tsx    → GET BY ID
      ├── update/
      │     └── page.tsx    → UPDATE
      └── delete/
            └── page.tsx    → DELETE
```

STEP 1 — Prisma Schema (prisma/schema.prisma)

```
model Student {
  id      Int    @id @default(autoincrement())
  name    String
  email   String
  age     Int
  gender  String
}
```

STEP 2 — GET ALL (app/student/page.tsx)

```
import { PrismaClient } from "@prisma/client";

export default async function Page() {
  const prisma = new PrismaClient();

  const data = await prisma.student.findMany();

  return (
    <ul>
      {data.map((stu: any) => (
        <li key={stu.id}>
          {stu.name} | {stu.email} | {stu.age} | {stu.gender}
        </li>
      ))}
    </ul>
  );
}
```

STEP 3 — CREATE (app/student/create/page.tsx)

Nextjs - CRUD

```
import { PrismaClient } from "@prisma/client";

export default async function Page() {
  const prisma = new PrismaClient();

  await prisma.student.create({
    data: {
      name: "Rahul",
      email: "rahul@gmail.com",
      age: 21,
      gender: "Male",
    },
  });

  return <h2>Student Inserted</h2>;
}
```

STEP 4 — GET BY ID (app/student/[id]/page.tsx)

```
import { PrismaClient } from "@prisma/client";

export default async function Page({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const prisma = new PrismaClient();
  const { id } = await params;

  const student = await prisma.student.findUnique({
    where: { id: Number(id) },
  });

  return (
    <div>
      <h2>Student Detail</h2>
      <p>{student?.name}</p>
      <p>{student?.email}</p>
      <p>{student?.age}</p>
      <p>{student?.gender}</p>
    </div>
  );
}
```

Nextjs - CRUD

STEP 5 — UPDATE (app/student/update/page.tsx)

```
import { PrismaClient } from "@prisma/client";

export default async function Page() {
  const prisma = new PrismaClient();

  await prisma.student.update({
    where: { id: 1 },
    data: {
      name: "Updated Name",
      email: "updated@gmail.com",
    },
  });

  return <h2>Student Updated</h2>;
}
```

STEP 6 — DELETE (app/student/delete/page.tsx)

```
import { PrismaClient } from "@prisma/client";

export default async function Page() {
  const prisma = new PrismaClient();

  await prisma.student.delete({
    where: { id: 1 },
  });

  return <h2>Student Deleted</h2>;
}
```